



LinkAid

Julia Spanopoulos
RM: 566754

Julia Valério
RM: 568275

Sumário

1. Objetivo e escopo do projeto.....	2
2. Principais funcionalidades implementadas.....	2
3. Métodos com lógica de negócio	2
3.1 Criação de ticket e protocolo	2
3.2 Atualização de status com regra de fechamento	3
3.3 Remoção segura de contato.....	3
3.4 Triagem via Watson/WhatsApp	3
4. Tabela de Endpoints da API RESTful	5
5. Protótipo - prints das telas implementadas.....	6
6. Diagrama de Classes atualizado	9
Diagrama 1 – Visão Geral por Pacotes (Link direto).....	10
Diagrama 2 – Entidade do Domínio (Link direto).....	11
Diagrama 3 – Resource, BO e DAO – Fluxo Principal (Link direto)	12
Diagrama 4 - Pacotes De Suporte - Segurança, Exceções, DTO e Integração (Link direto)	13
7. Tabela MER (Links 1 e 2).....	14
8. Como rodar a aplicação.....	14
8.1 Banco de dados Oracle.....	14
8.2 Backend Java/Quarkus	14
8.3 Frontend React.....	15

1. Objetivo e escopo do projeto

O LinkAid é uma solução de gestão de atendimentos para a Turma do Bem. A aplicação centraliza contatos recebidos por canais digitais, cria e acompanha tickets, apoia a triagem com informações de IA, registra mensagens e notas internas, organiza dentistas voluntários e disponibiliza indicadores para acompanhamento operacional.

Nesta entrega Java, o escopo principal é o backend REST em Quarkus, com persistência de dados em Oracle e exposição dos recursos necessários ao frontend. O projeto também contém scripts de banco, frontend React e um adaptador opcional de WhatsApp/Watson, mas as regras de negócio permanecem centralizadas na API Java.

- [Link do Repositório do Projeto](#)

2. Principais funcionalidades implementadas

- Autenticação por token simples com controle de perfis ADMIN e COLABORADOR.
- CRUD de contatos com validação de documento, e-mail, tipo de contato e bloqueio de exclusão quando há tickets vinculados.
- CRUD de dentistas voluntários com validação de CRO único, especialidade obrigatória e status operacional.
- Gestão de tickets com filtros, criação, atualização, arquivamento, troca de status, atribuição de responsável e geração de protocolo.
- Registro e consulta de mensagens do ticket, notas internas e histórico por contato.
- Dashboard e relatórios com totais, agrupamentos por status/canal, últimos tickets e notificações.
- Endpoint de integração Watson/WhatsApp para receber mensagens, inferir prioridade/classificação, reutilizar ticket aberto ou criar novo ticket.
- Documentação técnica automática via OpenAPI/Swagger em /swagger-ui.

3. Métodos com lógica de negócio

3.1 Criação de ticket e protocolo

TicketBO.criarEntidade cria ou reutiliza o contato, resolve canal/prioridade/classificação, define status inicial, persiste o ticket e gera o protocolo definitivo.

Arquivo: backend/api/src/main/java/com/turmadobem/bo/TicketBO.java

```
Ticket criarEntidade(LinkAidDtos.TicketRequest request) {
    Contato contato = contatoBO.criarOuAtualizarContatoBasico(
        request.idContato(),
        request.nomeContato(),
        request.documentoContato(),
        request.emailContato(),
        request.telefoneContato(),
        request.tipoContatoCodigo(),
        request.cidadeContato(),
        request.ufContato(),
        null
    );
    Canal canal = buscarCanal(request.canalCodigo());
    Prioridade prioridade = buscarPrioridade(request.prioridadeCodigo());
    Classificacao classificacao = request.classificacaoCodigo() == null
        ? buscarClassificacao("GERAL")
        : buscarClassificacao(request.classificacaoCodigo());
    Usuario responsavel = request.idUsuarioResponsavel() == null
        ? null
        : buscarUsuarioAtivo(request.idUsuarioResponsavel());
    Dentista dentista = request.idDentistaResponsavel() == null
        ? null
        : dentistaBO.buscarDentistaAtivo(request.idDentistaResponsavel());
```

```

        LocalDateTime agora = LocalDateTime.now();
        Ticket ticket = new Ticket();
        ticket.setProtocolo("TMP-" + UUID.randomUUID());
        ticket.setContato(contato);
        ticket.setCanal(canal);
        ticket.setStatus(buscarStatus(responsavel == null ? "NOVO" :
"EM_ATENDIMENTO"));
        ticket.setPrioridade(prioridade);
        ticket.setClassificacao(classificacao);
        ticket.setUsuarioResponsavel(responsavel);
        ticket.setDentistaResponsavel(dentista);
        ticket.setAssunto(normalizarObrigatorio(request.assunto(), "Assunto do ticket
e obrigatorio."));
        ticket.setDescricao(normalizarObrigatorio(request.descricao(), "Descricao do
ticket e obrigatoria."));
        ticket.setResumoIa(normalizarTexto(request.resumoIa()));
        ticket.setConfiancaIa(request.confiancaIa());
        ticket.setDataAbertura(agora);
        ticket.setDataAtualizacao(agora);

        ticketDAO.persist(ticket);
        ticketDAO.flush();
        ticket.setProtocolo(gerarProtocolo(ticket.getIdTicket(), agora));
        return ticket;
    }

```

3.2 Atualização de status com regra de fechamento

TicketBO.atualizarStatus chama aplicarStatus. Quando o status é final (RESOLVIDO, CANCELADO ou ARQUIVADO), a data de fechamento é preenchida; caso contrário, ela é removida.

Arquivo: backend/api/src/main/java/com/turmadobem/bo/TicketBO.java

```

public LinkAidDtos.TicketResponse atualizarStatus(Long id, String statusCodigo) {
    Ticket ticket = buscarEntidade(id);
    aplicarStatus(ticket, statusCodigo);
    ticket.setDataAtualizacao(LocalDateTime.now());
    return ApiMapper.ticket(ticket, mensagemDAO.listarPorTicket(id));
}

```

3.3 Remoção segura de contato

ContatoBO.remove impede a exclusão de contatos que já possuem tickets vinculados, preservando a integridade funcional e o histórico de atendimento.

Arquivo: backend/api/src/main/java/com/turmadobem/bo/ContatoBO.java

```

public void remover(Long id) {
    Contato contato = buscarEntidade(id);
    long tickets = ticketDAO.count("contato.idContato = ?1", id);
    if (tickets > 0) {
        throw new BusinessException("Nao e possivel remover contato com tickets
vinculados.");
    }
    contatoDAO.delete(contato);
}

```

3.4 Triagem via Watson/WhatsApp

WebhookBO.criarTicketViaWatson valida a mensagem, infere prioridade e classificação, reaproveita ticket aberto pelo telefone quando existir e registra o evento de integração.

Arquivo: backend/api/src/main/java/com/turmadobem/bo/WebhookBO.java

```
public LinkAidDtos.WebhookTicketResponse
criarTicketViaWatson(LinkAidDtos.WebhookTicketRequest request) {
    if (request.body() == null || request.body().isBlank()) {
        throw new BusinessException("Corpo da mensagem do webhook e
obrigatorio.");
    }

    String prioridade = primeiroValor(request.prioridadeCodigo(),
inferirPrioridade(request.body()));
    String classificacao = primeiroValor(request.classificacaoCodigo(),
inferirClassificacao(request.body(), request.intent()));
    String origem = primeiroValor(request.origem(),
"WATSON_SANDBOX").toUpperCase();
    String nome = primeiroValor(request.nome(), request.from(), "Contato
WhatsApp");
    boolean encaminharHumano = deveEncaminharHumano(request, classificacao);
    Ticket ticketAberto = ticketDAO.buscarAbertoPorTelefone(request.from());
    if (ticketAberto != null) {
        boolean atendimentoHumanoIniciado =
ticketMensagemDAO.existeAtendentePorTicket(ticketAberto.getIdTicket());
        ticketBO.registrarMensagem(ticketAberto, "CONTATO", request.body(),
null);
        if (!atendimentoHumanoIniciado) {
            ticketBO.registrarMensagem(ticketAberto, "IA",
respostaIa(request.respostaIa(), ticketAberto.getProtocolo(), encaminharHumano),
null);
        }
        registrarEvento(ticketAberto, request, origem, "PROCESSADO");
        return respostaWebhook(ticketBO.buscar(ticketAberto.getIdTicket()),
!atendimentoHumanoIniciado);
    }

    LinkAidDtos.TicketRequest ticketRequest = new LinkAidDtos.TicketRequest(
        null,
        nome,
        null,
        null,
        request.from(),
        primeiroValor(request.tipoContatoCodigo(), "SOLICITANTE"),
        null,
        null,
        canalPorOrigem(origem),
        prioridade,
        classificacao,
        null,
        null,
        tituloPorClassificacao(classificacao),
        request.body(),
        "Triagem Watson: " + tituloPorClassificacao(classificacao) + ".
Prioridade " + prioridade + ".",
        request.confiancaIa() == null ? BigDecimal.valueOf(90) :
request.confiancaIa()
    );

    Ticket ticket = ticketBO.criarEntidade(ticketRequest);
    ticketBO.registrarMensagem(ticket, "CONTATO", request.body(), null);
    ticketBO.registrarMensagem(ticket, "IA", respostaIa(request.respostaIa(),
ticket.getProtocolo(), encaminharHumano), null);
    registrarEvento(ticket, request, origem, "PROCESSADO");

    return respostaWebhook(ticketBO.buscar(ticket.getIdTicket()), true);
}
```

4. Tabela de Endpoints da API RESTful

Base local padrão: <http://localhost:8080>

Verbo	URI	Sucesso	Erros esperados
POST	/auth/login	200	400, 403, 500
GET	/auth/me	200	401, 403, 500
GET	/tickets	200	401, 403, 500
GET	/tickets/{id}	200	401, 403, 404, 500
POST	/tickets	201	400, 401, 403, 500
PUT	/tickets/{id}	200	400, 401, 403, 404, 500
PUT	/tickets/{id}/status	200	400, 401, 403, 404, 500
PUT	/tickets/{id}/responsavel	200	400, 401, 403, 404, 500
DELETE	/tickets/{id}	200	401, 403, 404, 500
POST	/tickets/{id}/liberar-telefone-teste	200	400, 401, 403, 404, 500
GET	/tickets/{id}/mensagens	200	401, 403, 404, 500
POST	/tickets/{id}/mensagens	201	400, 401, 403, 404, 500
GET	/tickets/{id}/notas-internas	200	401, 403, 404, 500
POST	/tickets/{id}/notas-internas	201	400, 401, 403, 404, 500
GET	/contatos	200	401, 403, 500
GET	/contatos/{id}	200	401, 403, 404, 500
GET	/contatos/{id}/historico	200	401, 403, 404, 500
POST	/contatos	201	400, 401, 403, 500
PUT	/contatos/{id}	200	400, 401, 403, 404, 500
DELETE	/contatos/{id}	204	400, 401, 403, 404, 500
GET	/dentistas	200	401, 403, 500
GET	/dentistas/{id}	200	401, 403, 404, 500
POST	/dentistas	201	400, 401, 403, 500
PUT	/dentistas/{id}	200	400, 401, 403, 404, 500
DELETE	/dentistas/{id}	204	400, 401, 403, 404, 500
GET	/usuarios?ativos=true	200	401, 403, 500
GET	/dashboard	200	401, 403, 500
GET	/dashboard/resumo	200	401, 403, 500
GET	/dashboard/tickets-por-status	200	401, 403, 500
GET	/dashboard/tickets-por-canal	200	401, 403, 500
GET	/dashboard/ultimos-tickets	200	401, 403, 500
GET	/dashboard/notificacoes	200	401, 403, 500
GET	/dominios	200	401, 403, 500
GET	/dominios/tipos-contato	200	401, 403, 500
GET	/dominios/canais	200	401, 403, 500
GET	/dominios/status-ticket	200	401, 403, 500
GET	/dominios/prioridades	200	401, 403, 500
GET	/dominios/classifica??es	200	401, 403, 500
GET	/relatorios	200	401, 403, 500
GET	/relatorios/resumo	200	401, 403, 500
GET	/relatorios/tickets-por-status	200	401, 403, 500
GET	/relatorios/tickets-por-canal	200	401, 403, 500
GET	/relatorios/?ltimos-tickets	200	401, 403, 500
POST	/integracoes/watson/tickets	201	400, 500

Esses são **códigos de status HTTP**: a API usa eles para informar se a requisição deu certo ou qual tipo de erro aconteceu.

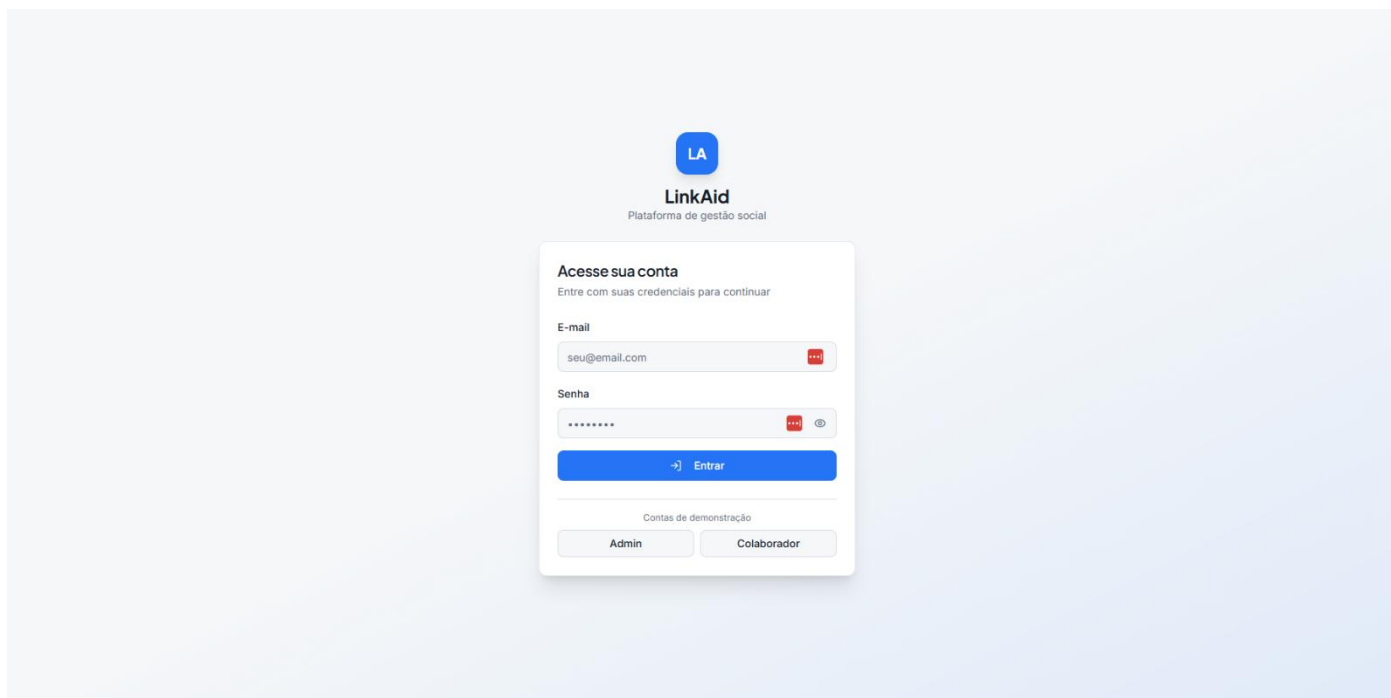
- **200 - OK**: requisição bem-sucedida. Usado em buscas, listagens, atualizações e ações que retornam dados.
- **201 - Created**: recurso criado com sucesso. Usado em POST que cria algo, como ticket, contato, dentista, mensagem ou nota.
- **204 - No Content**: operação bem-sucedida sem corpo de resposta. Usado em exclusões reais, como remover contato/dentista.

- **400 - Bad Request:** erro de validação ou regra de negócio. Exemplo: campo obrigatório ausente, status inválido, CRO duplicado, contato com ticket vinculado.
- **401 - Unauthorized:** falta de autenticação ou token inválido.
- **403 - Forbidden:** usuário autenticado, mas sem permissão para acessar o recurso.
- **404 - Not Found:** recurso não encontrado. Exemplo: ticket, contato ou dentista inexistente.
- **500 - Internal Server Error:** erro interno inesperado, incluindo falha não tratada ou problema de banco.

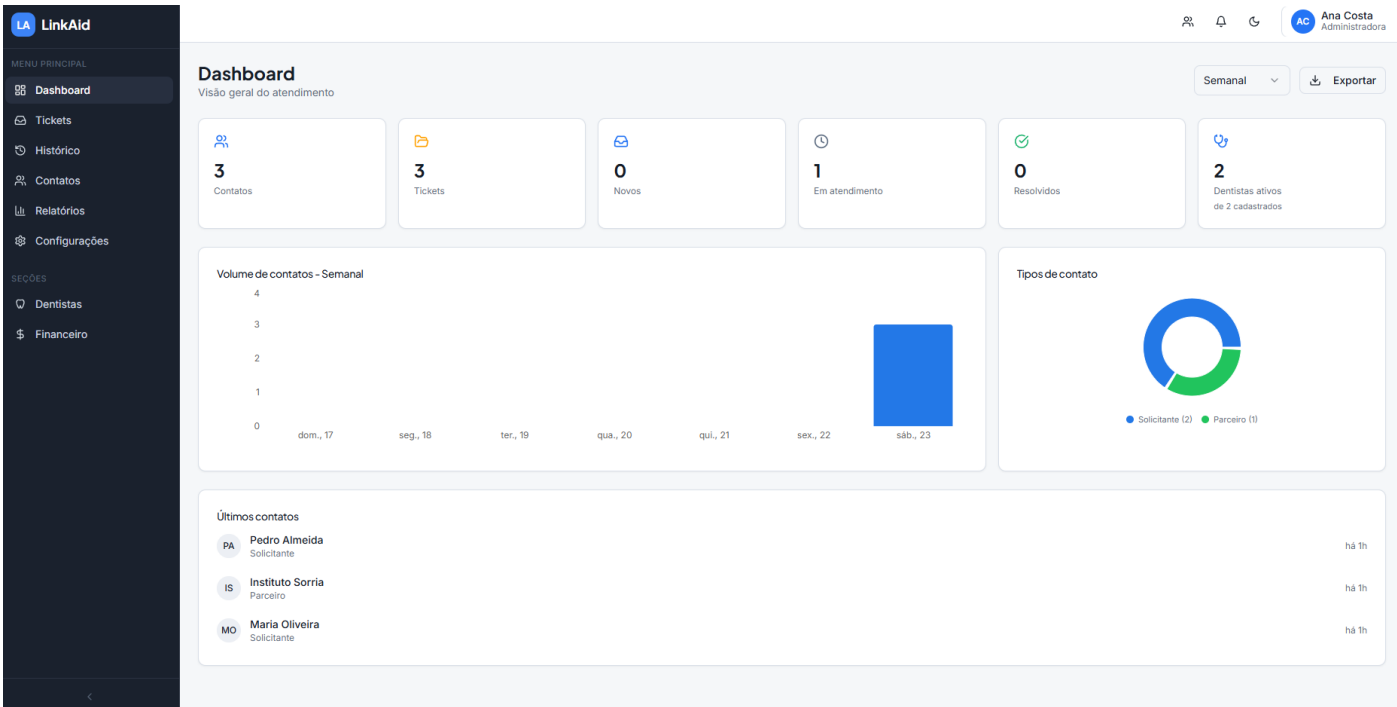
Incluí **só esses** porque são os que fazem sentido para o comportamento atual do projeto e aparecem no desenho real da API

5. Protótipo- prints das telas implementadas

As capturas abaixo foram obtidas do frontend local, renderizado em navegador headless, com dados de demonstração compatíveis com os DTOs da API Java. Em execução integrada, os dados são carregados do Oracle pela API Quarkus.



Tela de login com acesso por credenciais e contas de demonstração.



LA LinkAid

MENU PRINCIPAL

Dashboard

Tickets

Histórico

Contatos

Relatórios

Configurações

SEÇÕES

Dentistas

Financeiro

← Voltar

Ticket

TKT-20260514-001

14/05/2026, 09:00

Abertura

Remetente

MO

Maria Oliveira

+5511999990000

Tipo

Solicitante

Local

Sao Paulo, SP

🔒

Liberar numero para teste

Inteligência IA

Classificação

Saúde

Resumo

Solicitante buscando informacoes sobre tratamento gratuito.

Confiança

93%

Gerenciamento

Prioridade

Alta

Status

Em atendimento

Responsável pelo Atendimento

Carlos Silva — Colaborador

Ola, gostaria de saber sobre tratamento odontologico gratuito.

09:00

IA

Classificacao automatica: SAUDE. Prioridade: ALTA.

09:01

A você

Ola, Maria. Vamos verificar seu caso e orientar os proximos passos.

09:15

✎

Digite sua resposta...

📤

Gerenciamento

Prioridade

Alta

Crítica

✓ Alta

Média

Baixa

Canal

WhatsApp

Histórico

Dados Pessoais

Clínico

Nenhum ticket anterior para este CPF

Histórico

Dados Pessoais

Clínico

Nome

Maria Oliveira

CPF/CNPJ

123.456.789-00

E-mail

maria@email.com

Telefone

(55) 11999-9900

Tipo

Solicitante

Cidade

Sao Paulo

Estado

SP

Salvar Dados do Contato

Histórico

Dados Pessoais

Clínico

Descrição do Procedimento

Descreva o procedimento realizado...

Medicamentos Prescritos

Liste os medicamentos prescritos...

Histórico de Cirurgias / Intervenções

Registre cirurgias ou intervenções...

Detalhe do ticket com remetente, IA, gerenciamento, chat e ações de atendimento.

LA LinkAid

MENU PRINCIPAL

- Dashboard
- Tickets
- Histórico
- Contatos
- Relatórios
- Configurações

SEÇÕES

- Dentistas
- Financeiro

Criar Ticket
Cadastro manual de atendimento

Pessoa Física | Pessoa Jurídica

Dados do Paciente

Nome Completo: Nome completo do paciente CPF: 000.000.000-00

Telefone: (00) 00000-0000 Canal *: Seleccione o canal de entrada

Cidade: Cidade do remetente Estado: UF Tipo / Etiqueta: Solicitante

Assunto *: Assunto principal do ticket

Descrição: Descreva detalhadamente o atendimento...

Prioridade *: Seleccione a prioridade Responsável (Atendimento): Seleccione o responsável

Criar Ticket

Formulário de criação manual de ticket por pessoa física, jurídica ou dentista.

LA LinkAid

MENU PRINCIPAL

- Dashboard
- Tickets
- Histórico
- Contatos
- Relatórios
- Configurações

SEÇÕES

- Dentistas
- Financeiro

Dentistas
Cadastro e gestão de dentistas voluntários

+ Cadastrar Dentista

Q Buscar por nome, especialidade ou CRO... Status

Nome	Especialidade	Localização	Status	Vagas	Data de cadastro	Ações
Dr. Marcos Lima	Cirurgia	Belo Horizonte, MG	Ativo	0	23/05/2026	
Dra. Fernanda Costa	Ortodontia	Sao Paulo, SP	Ativo	0	23/05/2026	

Gestão de dentistas voluntários com busca, filtros, status e ações de comunicação.

6. Diagrama de Classes atualizado

O diagrama de classes representa a organização da API Java do LinkAid conforme a estrutura real do projeto. As classes foram agrupadas por pacote, separando as responsabilidades principais da aplicação: **model**, **dao**, **bo**, **resource**, **exception**, **security**, **integration** e **dto**.

Diagrama 1 – Visão Geral por Pacotes ([Link direto](#))

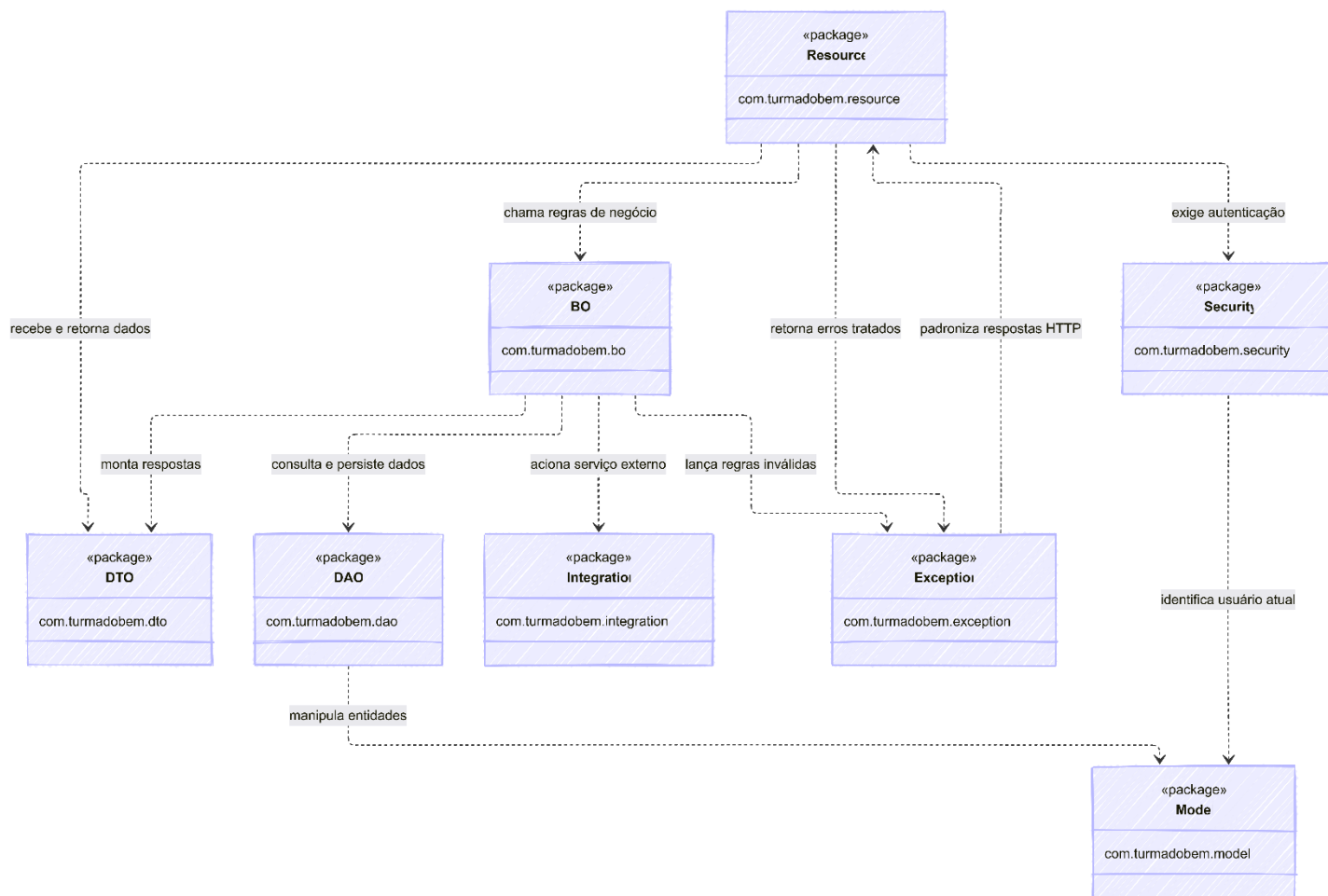


Diagrama 2 – Entidade do Domínio ([Link direto](#))

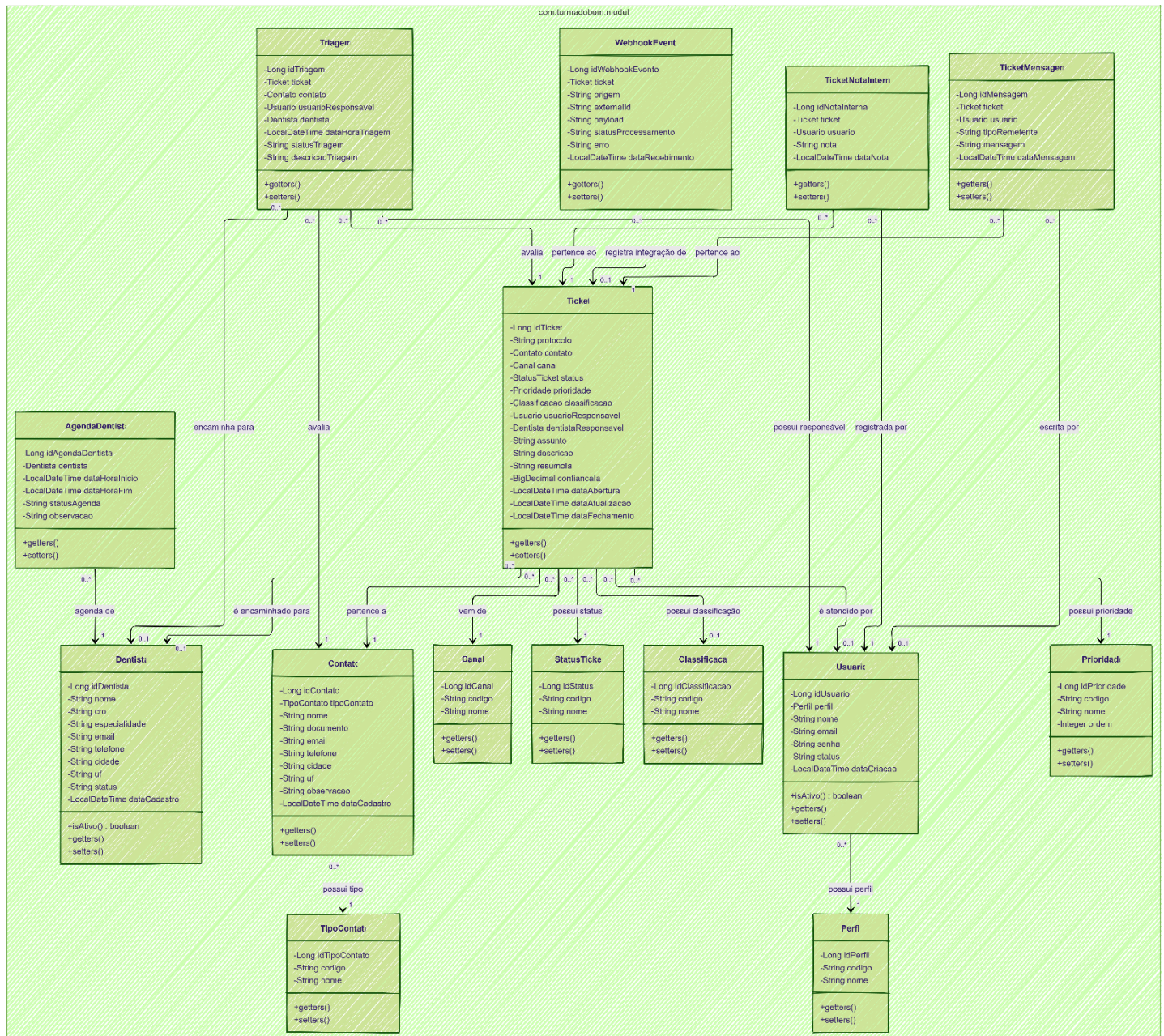


Diagrama 3 – Resource, BO e DAO – Fluxo Principal ([Link direto](#))

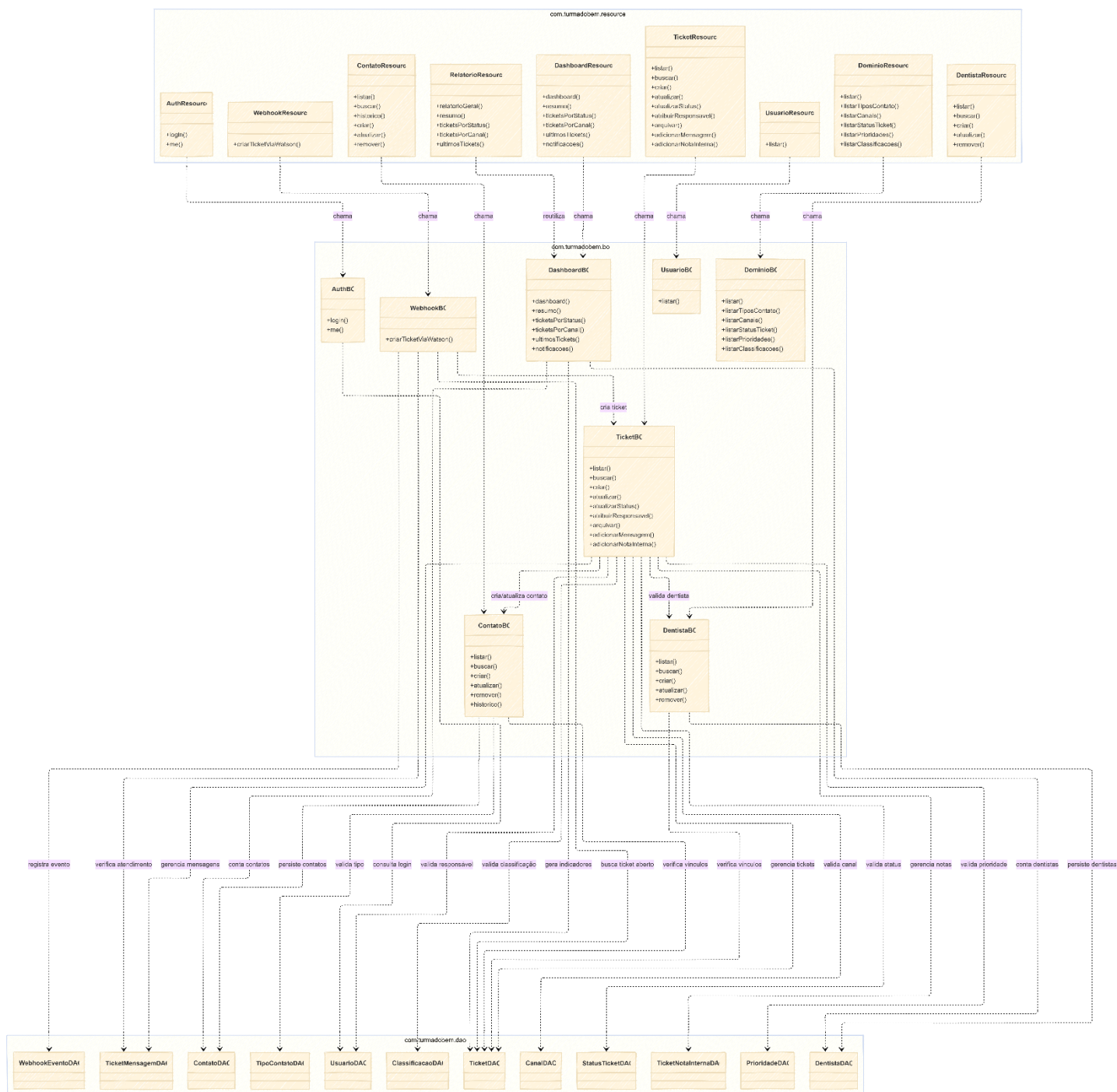
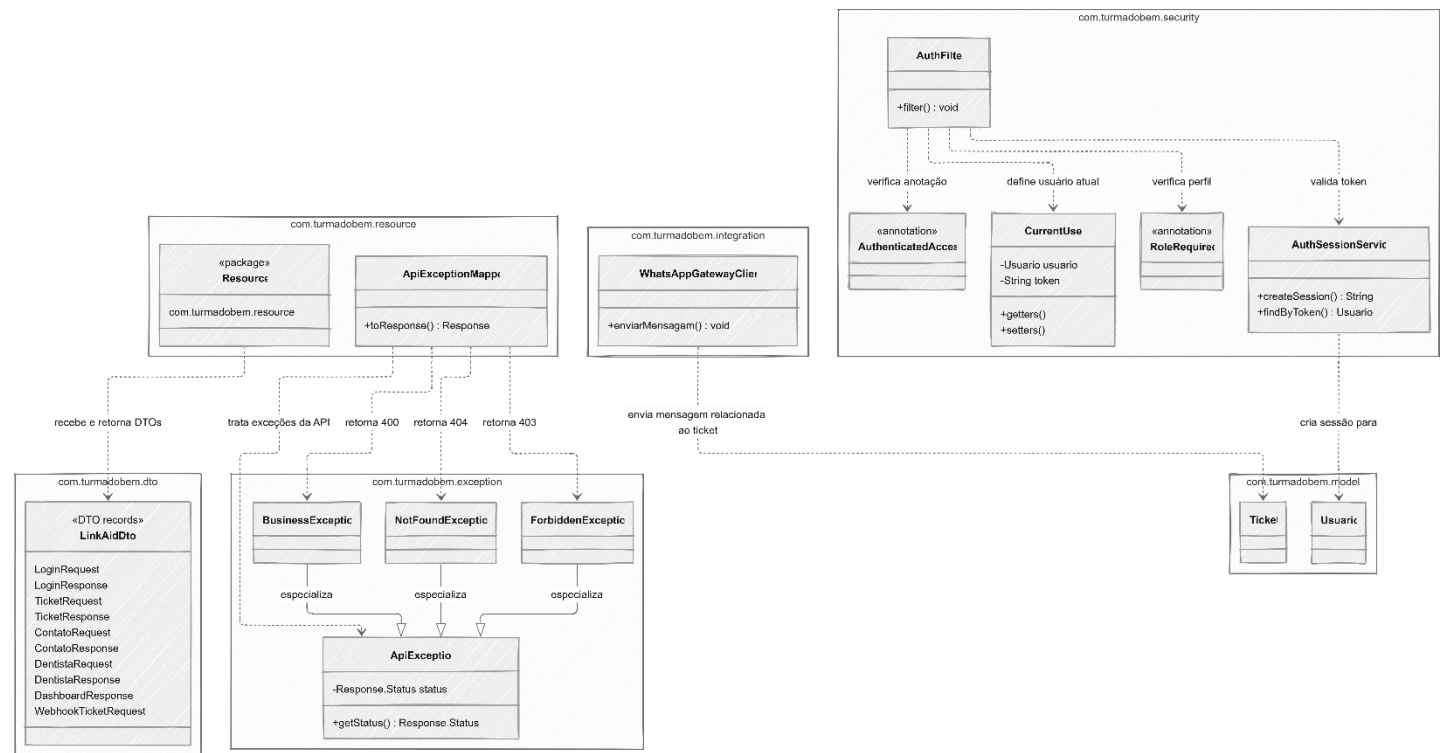


Diagrama 4- Pacotes De Suporte- Segurança, Exceções, DTO e Integração ([Link direto](#))



No pacote `com.turmadobem.model`, estão as entidades JPA que representam o modelo de domínio e as tabelas do banco de dados Oracle, como `Ticket`, `Contato`, `Dentista`, `Usuario`, `Canal`, `StatusTicket`, `Prioridade`, `Classificacao`, `TicketMensagem`, `TicketNotaInterna`, `Triagem`, `AgendaDentista` e `WebhookEvento`. Essas classes possuem atributos privados, construtor padrão, métodos getters e setters, além de relações de associação entre si.

As relações entre as entidades indicam como os objetos se conectam dentro do domínio. Por exemplo, um `Ticket` pertence a um `Contato`, possui um `Canal`, um `StatusTicket`, uma `Prioridade` e pode estar associado a um `Usuario` responsável e a um `Dentista` responsável. As classes `TicketMensagem`, `TicketNotaInterna` e `WebhookEvento` também se vinculam ao `Ticket`, registrando mensagens, observações internas e eventos de integração.

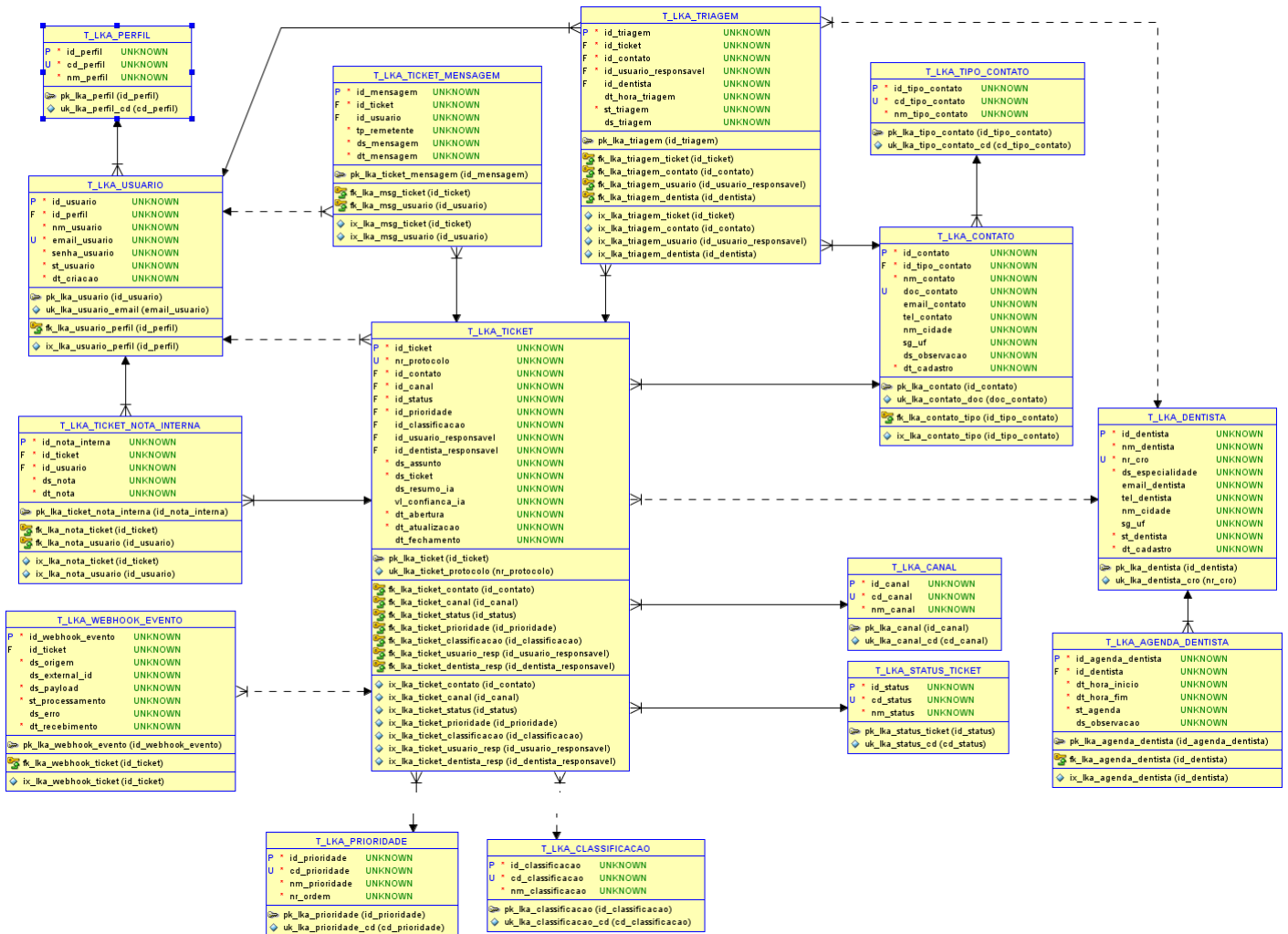
O pacote `com.turmadobem.dao` contém as classes responsáveis pelo acesso aos dados. Os DAOs utilizam `Panache Repository` para consultar e persistir as entidades, como `TicketDAO`, `ContatoDAO`, `DentistaDAO`, `UsuarioDAO` e os DAOs de domínio. Já o pacote `com.turmadobem.bo` concentra as regras de negócio, incluindo validações, criação e atualização de registros, geração de protocolo, arquivamento de tickets, controle de status, dashboard e processamento de integrações via Watson/WhatsApp.

O pacote `com.turmadobem.resource` contém os recursos REST da aplicação. Essas classes expõem os endpoints consumidos pelo frontend, como autenticação, tickets, contatos, dentistas, dashboard, relatórios, domínios, usuários e integrações. A camada `resource` chama a camada `bo`, mantendo a separação entre entrada HTTP e regras de negócio.

Também foram representados os pacotes de suporte da aplicação. Em `exception`, ficam as exceções customizadas e o tratamento centralizado de erros. Em `security`, estão as classes responsáveis pela autenticação, sessão, usuário atual e controle de perfil. Em `integration`, está o cliente de comunicação com o gateway de WhatsApp. Por fim, o pacote `dto` agrupa os objetos de entrada e saída usados na comunicação entre API e frontend.

Dessa forma, o diagrama evidencia a separação em camadas, as principais entidades do domínio, as dependências entre os pacotes e os relacionamentos necessários para o funcionamento integrado da aplicação.

7. Tabela MER (Links [1](#) e [2](#))



8. Como rodar a aplicação

O site já está rodando e completamente conectado com API hospedada no Render, banco de dados Oracle no banco de dados da FIAP e Frontend hospedado no Vercel. [Link para o site](#).

8.1 Banco de dados Oracle

1. Abrir o Oracle SQL Developer ou ferramenta equivalente.
2. Executar o script database/01_linkaid_mvp_oracle.sql.
3. Confirmar que as tabelas T_LKA_* foram criadas e que os inserts de demonstração foram aplicados.

Usuários de demonstração criados pelo script:

Perfil	E-mail	Senha
ADMIN	admin@linkaid.com	admin123
COLABORADOR	colab@linkaid.com	colab123

8.2 Backend Java/Quarkus

Definir as variáveis de ambiente antes de iniciar a API. Exemplo em PowerShell:

```
$env:ORACLE_USERNAME="seu_usuario_oracle"
$env:ORACLE_PASSWORD="sua_senha_oracle"
$env:ORACLE_JDBC_URL="jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL"
cd backend\api
.\mvnw.cmd quarkus:dev
```

URLs úteis:

- API local: <http://localhost:8080>
- Swagger UI: <http://localhost:8080/swagger-ui>
- OpenAPI JSON/YAML: <http://localhost:8080/openapi>

8.3 Frontend React

```
cd frontend
npm install
$env:VITE_API_URL="http://localhost:8080"
npm run dev
```

Acessar o frontend em <http://localhost:5173> e autenticar com um dos usuários de demonstração.